Experiment 6

Student Name: Nitin **UID: 23BCS12270**
Branch: CSE **Section/Group: KRG 3-A**
Semester: 6th **Date of Performance:03/03/2026**
Subject Name: Full Stack Development – II **Subject Code: 23CSH-309**

1. **Aim**: To design and implement a **Spring Boot REST API using Layered Architecture**, applying **Inversion of Control (IoC)** and **Dependency Injection (DI)** to build a structured backend for the **HealthHub application**, including CRUD operations, DTO mapping, validation, and global exception handling.

2. **Objective**:

• Understand the role of **Spring Boot in backend development**
• Configure and run a **basic Spring Boot application**
• Apply **IoC (Inversion of Control)** and **Dependency Injection**
• Use Spring annotations such as **@Autowired, @Service, and @Repository**
• Implement **Layered Architecture (Controller – Service – Repository)**
• Develop **RESTful APIs using Spring Boot**
• Perform **CRUD operations (Create, Read, Update, Delete)** using HTTP methods
• Implement **DTO mapping** to separate internal data models from API responses
• Apply **validation using Jakarta Validation annotations** like `@NotNull`, `@Email`
• Implement **Global Exception Handling using @ControllerAdvice**
• Build backend services for **HealthHub patient management system**.

3. **Implementation / Code:**

- **Tools & Technologies Used:-**

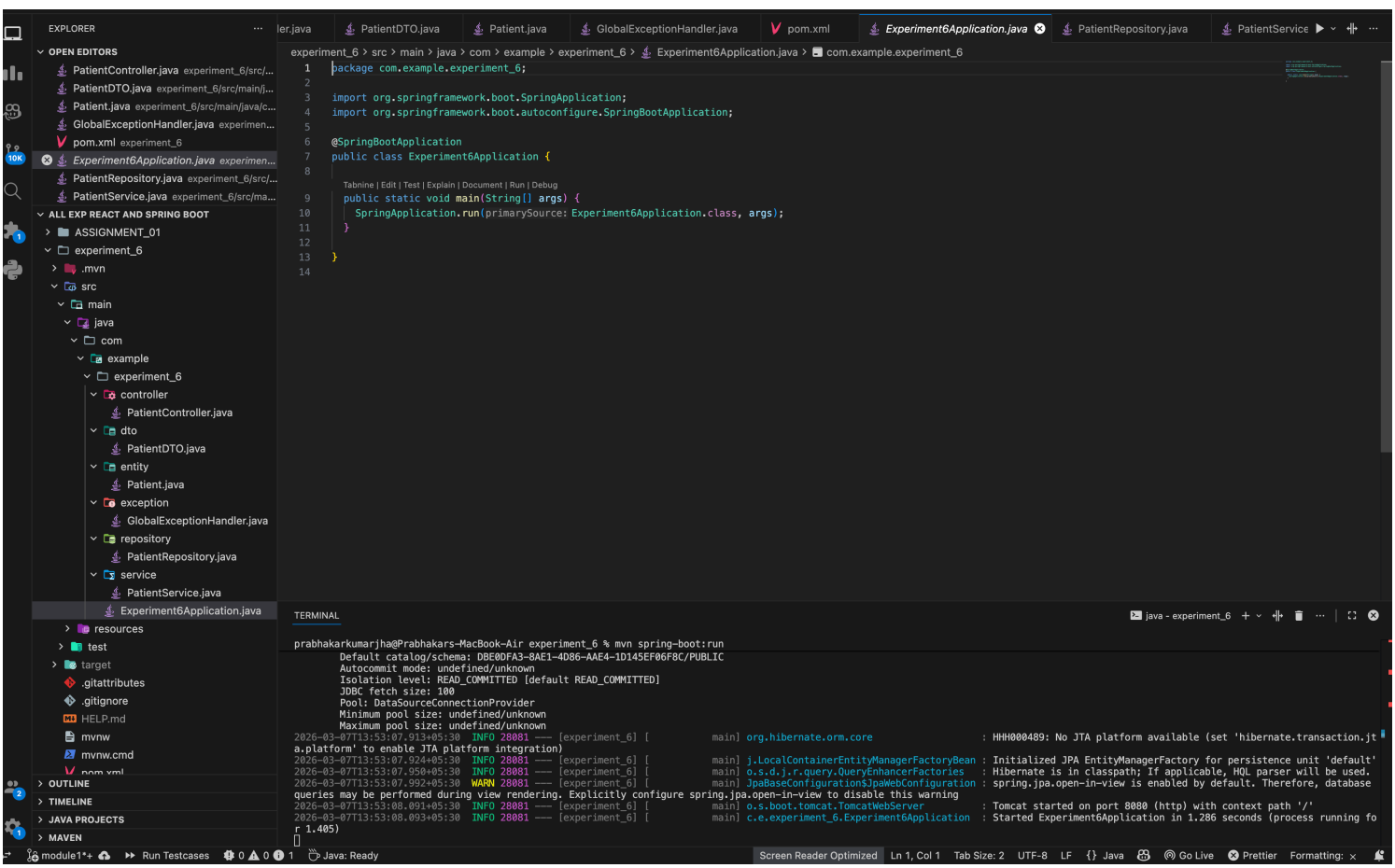
  • Java
  • Spring Boot
  • Spring Web
  • Spring Data JPA
  • Maven
  • VS Code
  • Postman (API testing)
  • H2 Database

- **Implementation Description:-**

• A **Spring Boot application named HealthHub** is created to manage patient data.
• The project follows **Layered Architecture**, separating logic into different layers:

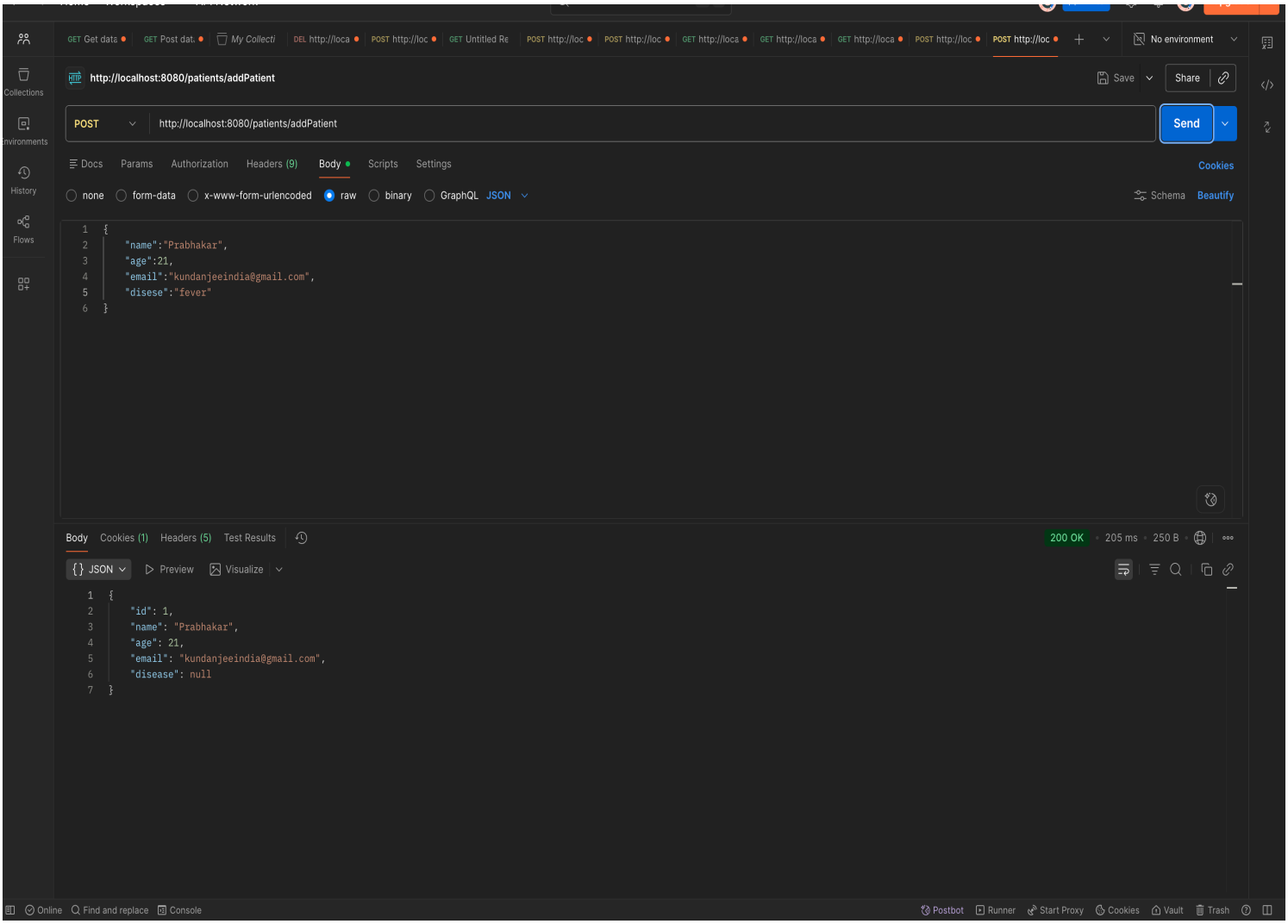
- **Controller Layer**
  Handles HTTP requests and responses using REST APIs.
- **Service Layer**
  Contains business logic for managing patients.
- **Repository Layer**
  Communicates with the database using Spring Data JPA.
- **Entity Layer**
  Represents database tables.
- **DTO Layer**
  Transfers patient data between API and application.
- **Exception Layer**
  Handles errors globally using `@ControllerAdvice`.

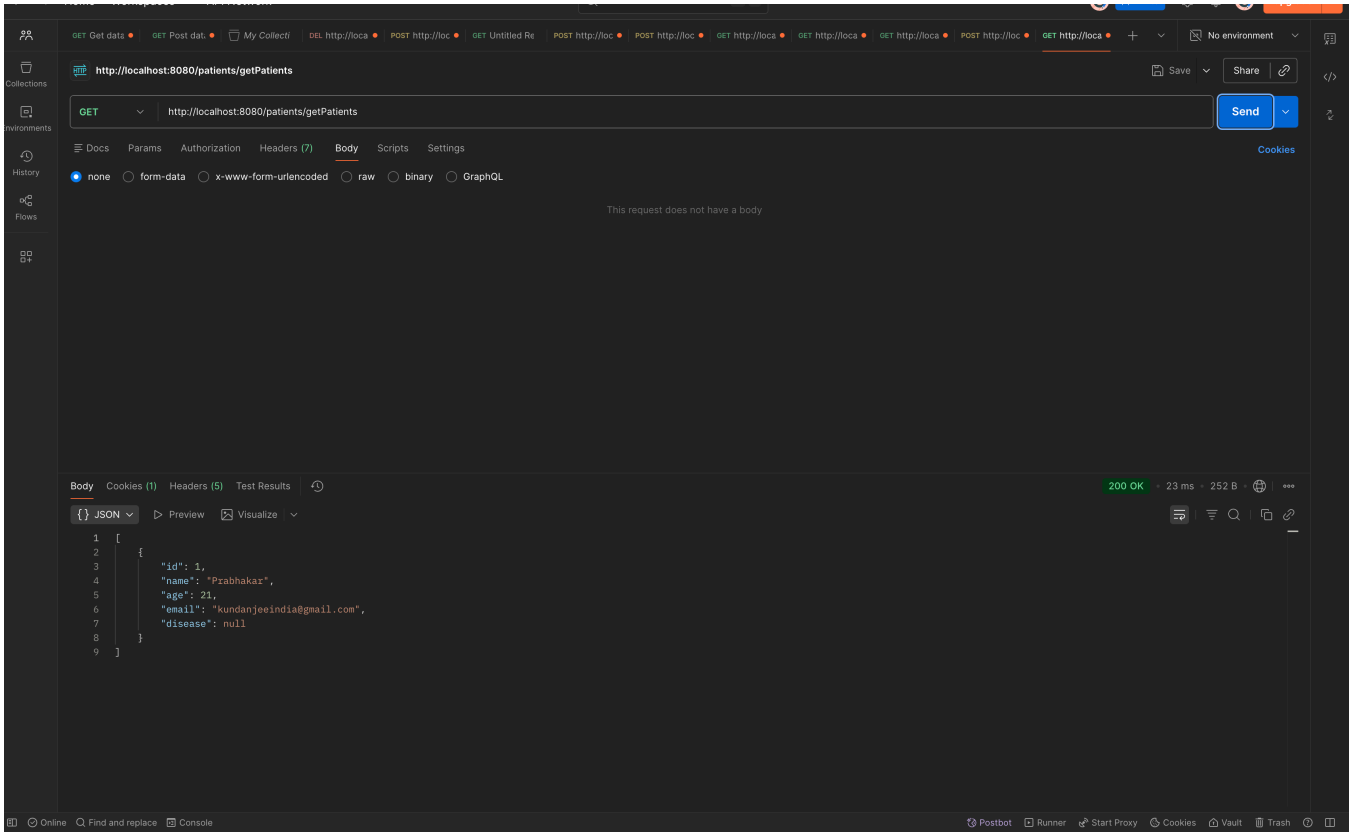
▪ Project Structure:-

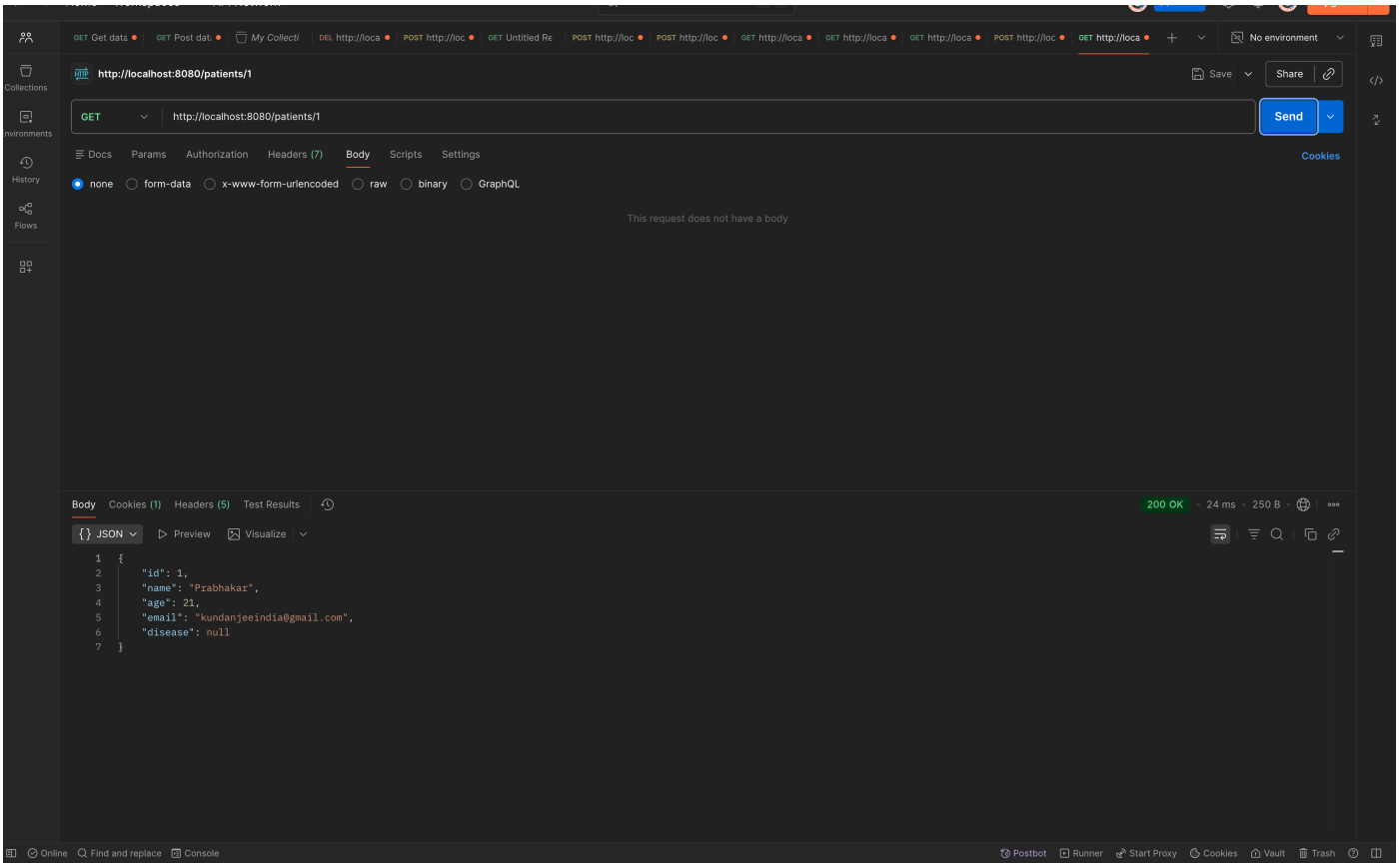


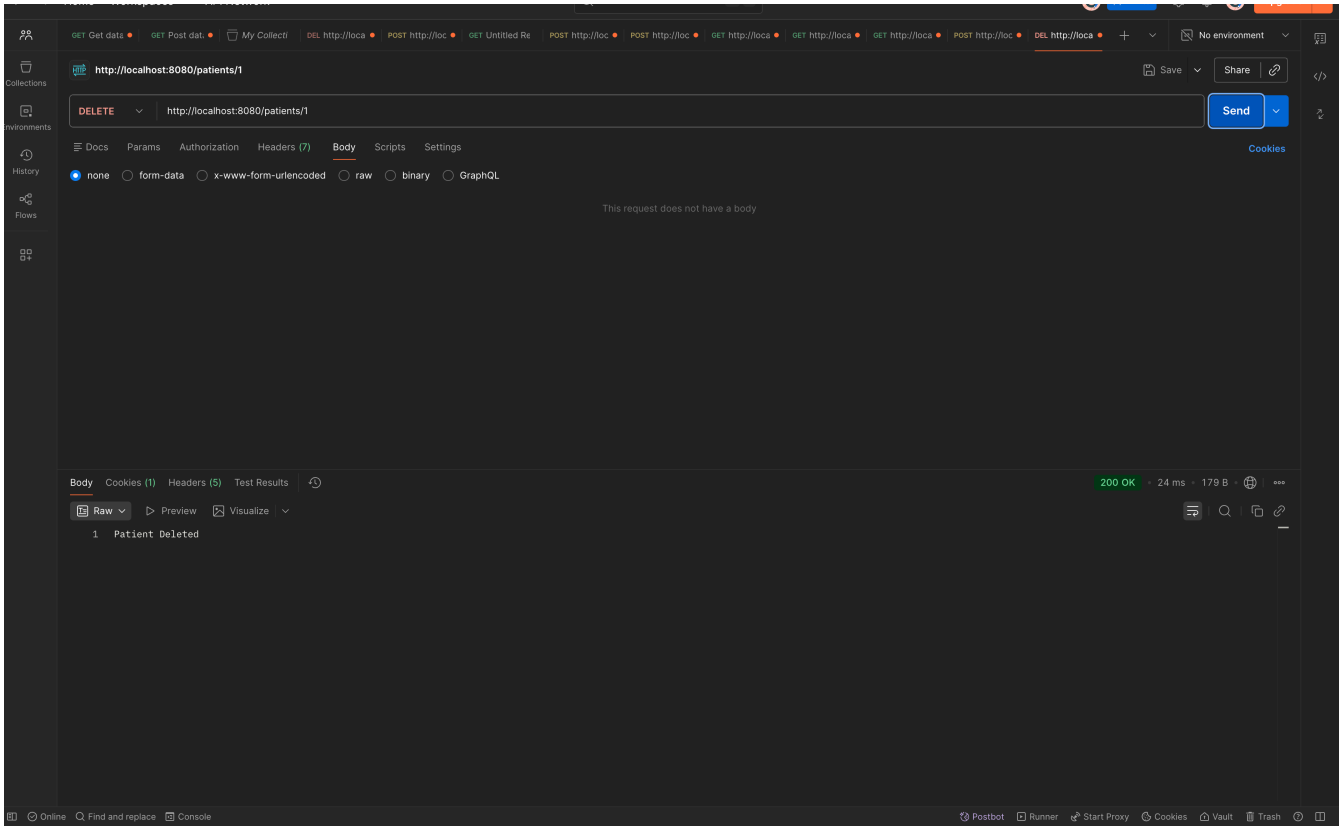
4. Output:

- Spring Boot application started successfully on **localhost:8080**
- POST API successfully added patient records
- GET API returned the list of patients stored in the database
- GET by ID API returned specific patient details
- DELETE API successfully removed patient data
- Postman confirmed successful API responses with **Status 200 OK**
- Layered architecture ensured separation of concerns and clean backend design




http://localhost:8080/patients/getPatients
GET
http://localhost:8080/patients/getPatients
Send
Body
Cookies (1)
Headers (5)
Test Results
200 OK
23 ms
252 B
JSON
Preview
Visualize
[
{
"id": 1,
"name": "Prabhakar",
"age": 21,
"email": "kundanjeeindia@gmail.com",
"disease": null
}
]


http://localhost:8080/patients/1
GET
http://localhost:8080/patients/1
Send
Body
Cookies (1)
Headers (5)
Test Results
200 OK
24 ms
250 B
JSON
Preview
Visualize
{
"id": 1,
"name": "Prabhakar",
"age": 21,
"email": "kundanjeeindia@gmail.com",
"disease": null
}



5. Learning Outcomes (What I Have Learnt):

- Understanding of **Spring Boot backend development**
- Implementation of **Layered Architecture (Controller, Service, Repository)**
- Usage of **Dependency Injection in Spring Boot**
- Development of **RESTful APIs using HTTP methods**
- Implementation of **CRUD operations in backend applications**
- Understanding of **DTO mapping for API communication**
- Applying **validation annotations for data integrity**
- Handling errors globally using **Spring Exception Handling**
- Testing APIs using **Postman**